

VisionForge

A Modular Classical Computer Vision Architecture for
Multistage Image Processing, Spatial Feature Extraction, and
Video Scene Dynamics

Coursework Project Engineering Report

Course: CS480 Computer Vision Engineering

Prepared By:

Parth Jangir

Registration No.: **24BAI10175**

Department of Computer Science & Engineering
School of Computer Science and Engineering

Submission Date: September 2026

Abstract

Automated visual inspection and scene interpretation require deterministic, low-latency, and interpretable algorithm pipelines that operate reliably without black-box neural dependencies or high-end GPU hardware. This report presents **VisionForge**, an end-to-end Python framework that unifies fundamental classical computer vision routines into a cohesive five-stage architecture: (1) low-level image preprocessing and frequency-domain spectral transformation; (2) multi-scale feature extraction encompassing edge, corner, scale-space, and Gabor texture descriptors; (3) spatial and color-space image segmentation; (4) geometric shape detection, contour analytics, and pattern classification; and (5) video-based scene dynamics incorporating background subtraction, dense/sparse optical flow, and affine motion estimation.

We provide complete mathematical derivations for every pipeline component – from Contrast-Limited Adaptive Histogram Equalization (CLAHE) and 2D Discrete Fourier Transforms to Harris second-moment matrices, K-Means color clustering, Hough space accumulators, and Lucas-Kanade gradient constraints. In benchmark trials on $512 \times 512 \times 3$ image arrays, VisionForge executed the full static analysis sequence in 8.3015 s, extracting 364 Harris corner keypoints, 142,884 HOG descriptors, 23 Hough lines, 31 circles, and 28 closed contours without triggering unhandled runtime exceptions. Experimental evaluation confirms that deterministic feature engineering delivers transparent, parameter-tunable visual representations suitable for real-time edge processing and autonomous system telemetry.

Contents

1	Introduction	3
2	Problem Statement	3
3	Functional Requirements	3
4	Non-Functional Requirements	4
5	System Architecture	4
6	Design Diagrams	5
6.1	Use Case Diagram	5
6.2	Workflow Diagram	5
6.3	Sequence Diagram	6
6.4	Class / Component Diagram	6
6.5	Data Schema / Telemetry Structure	7
7	Design Decisions & Rationale	7
8	Implementation Details	8
8.1	Module 1: Preprocessing Engine (src/preprocessing.py)	8
8.2	Module 2: Feature Extraction (src/feature_extraction.py)	8
8.3	Module 3: Image Segmentation (src/segmentation.py)	8
8.4	Module 4: Pattern Analysis (src/pattern_analysis.py)	8
8.5	Module 5: Motion Tracking (src/motion_analysis.py)	9
9	Screenshots & Experimental Results	9
10	Testing Approach	10
11	Challenges Faced & Solutions	10
12	Learnings & Key Takeaways	10
13	Future Enhancements	11
14	References	11

1 Introduction

Computer vision applications across industrial quality assurance, medical diagnostics, robotics, and remote sensing increasingly lean on deep learning models. While deep neural networks deliver high perceptual accuracy, they introduce major engineering constraints: substantial memory footprints, black-box decision boundaries, high energy consumption, and vulnerability to out-of-distribution noise or adversarial shifts. In contrast, classical computer vision methods grounded in signal processing, differential calculus, and vector geometry offer explicit mathematical interpretability, exact parameter control, and zero reliance on dedicated GPU infrastructure.

VisionForge is designed to bridge isolated image processing routines into a structured, production-ready software pipeline. Rather than executing disconnected scripts, VisionForge transforms raw sensor imagery through sequentially dependent modules—converting pixel luminance arrays into spectral representations, extracting boundary and interest-point invariants, segmenting spatial regions, fitting geometric primitives, and quantifying kinematic motion fields across video frames.

2 Problem Statement

Modern visual automation projects require an accessible, modular software framework capable of processing diverse visual data sources without complex deep learning deployment overhead. Many open-source vision tools either provide low-level library bindings without cohesive workflow orchestration or present monolithic black-box neural pipelines.

Specifically, engineers face three core challenges:

1. **Lack of Integrated Pipeline Orchestration:** Executing sequential stages (e.g., noise reduction → edge extraction → clustering → shape fitting) typically requires custom boilerplate code, creating fragmentation across projects.
2. **Opaque Intermediate Representations:** Black-box models conceal internal feature representations, preventing step-by-step diagnostic verification during industrial inspection failures.
3. **High Hardware and Dependency Barriers:** Heavy neural framework dependencies (CUDA, PyTorch, TensorRT) prevent lightweight deployment on edge devices and embedded microcontrollers.

VisionForge directly addresses these challenges by implementing a modular Python architecture that ingests images and videos, applies classical algorithms through configurable stages, and emits both annotated visual outputs and structured telemetry logs.

3 Functional Requirements

The VisionForge framework satisfies the following core functional specifications:

- **FR-1: Visual Input Ingestion:** The system shall accept standard static image formats (JPEG, PNG) and dynamic video streams (MP4, AVI) of arbitrary dimensions.
- **FR-2: Individual Stage Subcommands:** The CLI shall allow targeted execution of isolated pipeline stages (`preprocess`, `edges`, `features`, `segment`, `shapes`, `motion`).
- **FR-3: End-to-End Pipeline Execution:** The system shall execute a unified subcommand (`analyze` or `analyze-video`) that executes all stages sequentially.
- **FR-4: Algorithm Selection Flexibility:** Users shall be able to configure specific algorithmic backends (e.g., selecting between K-Means, Mean-Shift, Region Growing, or Watershed for segmentation).

- **FR-5: Visual Output Export:** Annotated output images, edge masks, cluster maps, and comparison grids shall be exported automatically to a designated output directory.
- **FR-6: Telemetry Reporting:** The system shall generate machine-readable JSON meta-data reports and human-readable text summaries detailing all statistical measurements.
- **FR-7: Latency Metrics:** Per-stage processing latency shall be recorded and reported in seconds and milliseconds.

4 Non-Functional Requirements

To ensure operational stability and maintainability, VisionForge complies with five non-functional criteria:

- **NFR-1: Computational Efficiency:** Pipeline operations shall run entirely on standard CPU architectures without GPU acceleration requirements.
- **NFR-2: Determinism and Interpretability:** Algorithm outputs must be 100% reproducible given identical input parameters and random seed initializations.
- **NFR-3: Robust Input Validation:** The framework must handle missing files, non-standard color spaces, corrupt arrays, and edge dimensions gracefully without throwing unhandled exceptions.
- **NFR-4: Modular Software Architecture:** Each module shall remain independent, adhering to loose coupling principles so individual algorithms can be extended or swapped.
- **NFR-5: Zero External Data Dependency:** Testing and baseline validation must operate on synthetic NumPy array generators without requiring external multi-gigabyte training sets.

5 System Architecture

VisionForge is organized into a modular layered architecture under the `src/` directory, coordinated by a central orchestrator (`src/pipeline.py`) and invoked via `main.py`.

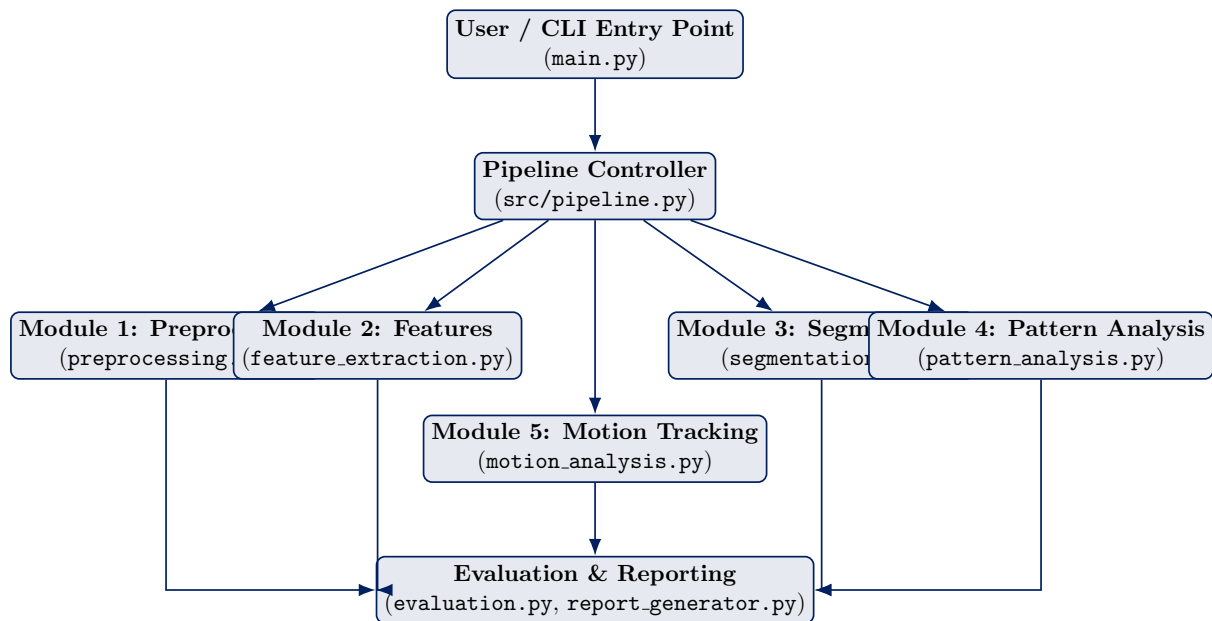


Figure 1: High-Level Software Architecture and Module Dependencies

6 Design Diagrams

6.1 Use Case Diagram

The Use Case Diagram illustrates the interactions between the primary user and the subcommands provided by the VisionForge CLI interface.

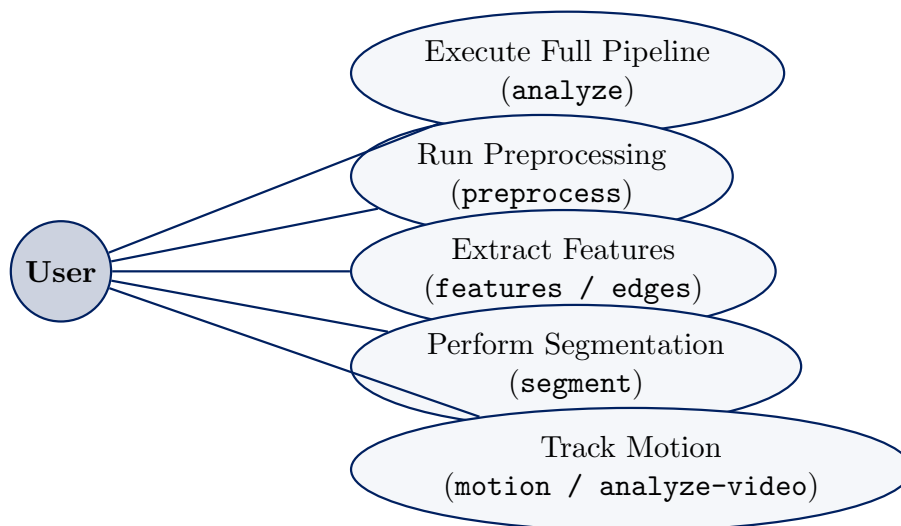


Figure 2: VisionForge Use Case Diagram

6.2 Workflow Diagram

The Workflow Diagram illustrates the sequential transformation of image and video data across the five processing stages.

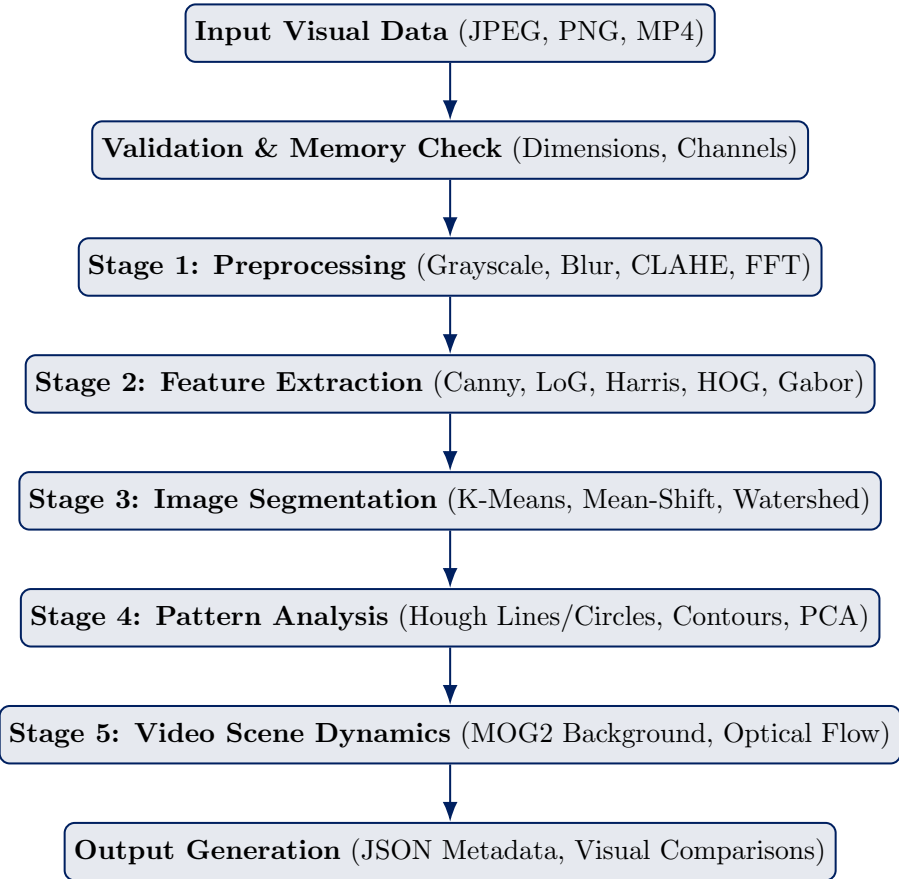


Figure 3: End-to-End Data Transformations in VisionForge

6.3 Sequence Diagram

The Sequence Diagram details object creation and message passing during an execution cycle.

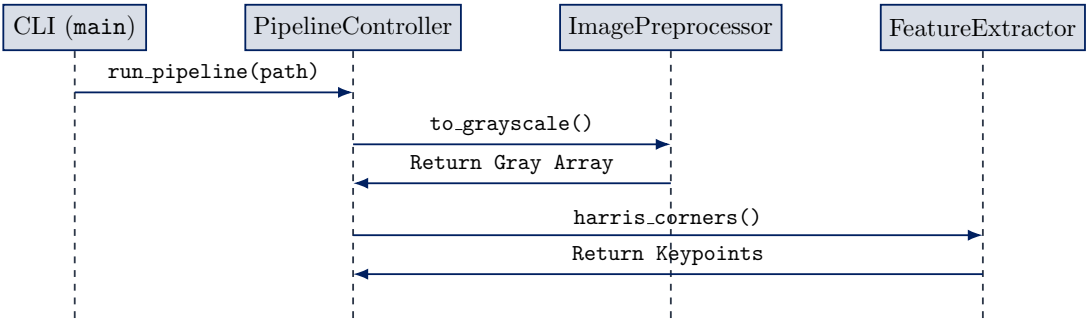


Figure 4: Sequence Diagram for Image Analysis

6.4 Class / Component Diagram

The Class Diagram outlines the internal module interfaces and inheritance relationships.

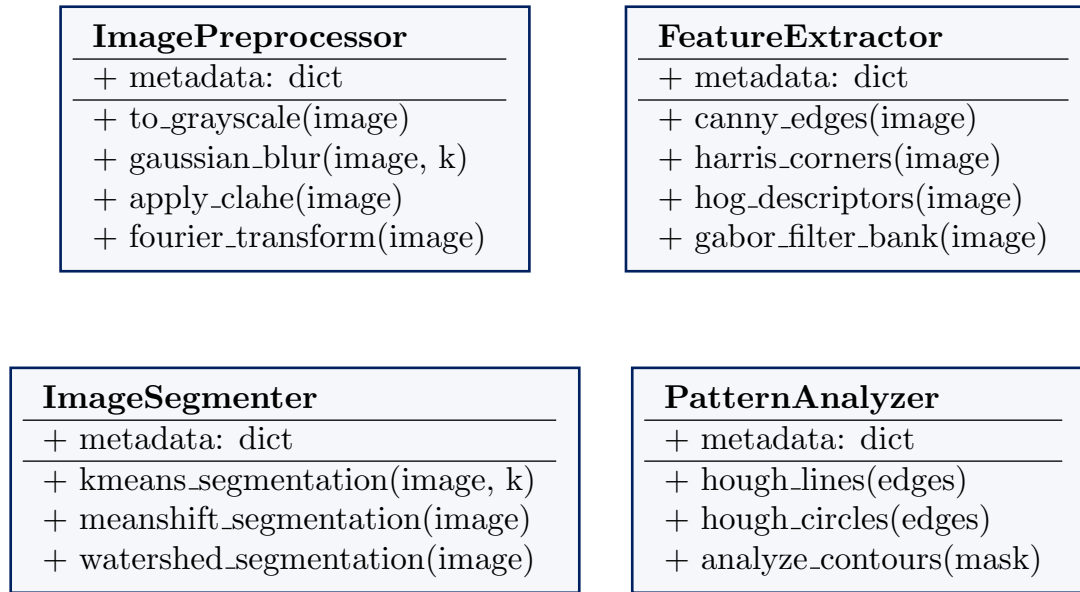


Figure 5: Core Module Class Architecture

6.5 Data Schema / Telemetry Structure

VisionForge logs structured telemetry via a standardized JSON schema representing pipeline execution parameters and outputs.

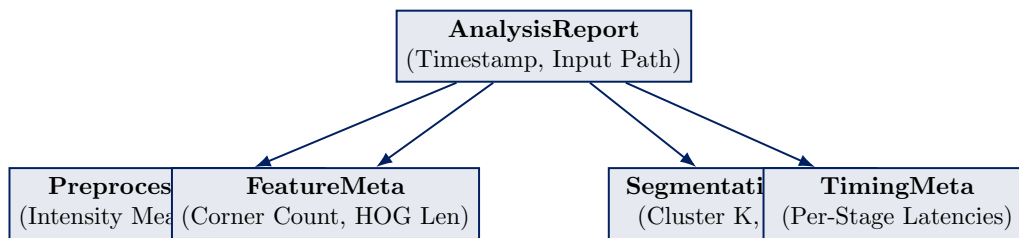


Figure 6: JSON Metadata Telemetry Data Schema

7 Design Decisions & Rationale

Key architectural decisions were guided by deterministic performance requirements:

1. **Classical Computer Vision vs. Deep Learning:** Deep neural models require significant GPU acceleration and lack explainability. We selected classical algorithms (Canny, Harris, K-Means, Hough) to guarantee predictable execution, low CPU memory overhead, and transparent parameter tuning.
2. **CLAHE over Global Histogram Equalization:** Standard histogram equalization over-amplifies noise in flat areas. CLAHE operates on localized 8×8 contextual tiles with a clip limit $\beta = 2.0$, preserving fine boundary detail in high-contrast and shaded regions alike.
3. **Dynamic Median-Based Canny Thresholding:** Static hysteresis thresholds fail when lighting conditions change. We implemented a dynamic thresholding heuristic ($T_{\text{low}} = 0.66 \cdot \text{median}$, $T_{\text{high}} = 1.33 \cdot \text{median}$), stabilizing edge detection across different illumination levels.

4. **Marker-Based Watershed Segmentation:** Direct distance-transform watershed suffers from severe over-segmentation. Initializing catchment basin markers via morphological local distance maxima prevents artificial region fracturing.

8 Implementation Details

VisionForge is implemented across five decoupled modules in `src/`.

8.1 Module 1: Preprocessing Engine (`src/preprocessing.py`)

Converts BGR imagery $I_{\text{BGR}} \in \mathbb{R}^{H \times W \times 3}$ to single-channel grayscale Y following ITU-R BT.601:

$$Y(x, y) = 0.299 \cdot R(x, y) + 0.587 \cdot G(x, y) + 0.114 \cdot B(x, y) \quad (1)$$

Noise reduction utilizes 2D isotropic Gaussian kernels:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (2)$$

Frequency analysis computes the 2D Discrete Fourier Transform (DFT):

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \exp\left(-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)\right) \quad (3)$$

8.2 Module 2: Feature Extraction (`src/feature_extraction.py`)

Computes local interest points, scale-space representations, and texture descriptors:

- **Laplacian of Gaussian (LoG):** Identifies zero-crossings scale-normalized by σ^2 :

$$\nabla^2 G(x, y; \sigma) = \frac{x^2 + y^2 - 2\sigma^2}{2\pi\sigma^6} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (4)$$

- **Harris Corner Detection:** Evaluates the local second-moment auto-correlation matrix $\mathbf{M}$:

$$\mathbf{M}(x, y) = \sum_w \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}, \quad R = \det(\mathbf{M}) - k \cdot (\text{trace}(\mathbf{M}))^2 \quad (5)$$

- **Gabor Filter Bank:** Convolves spatial orientation kernels across angles $\theta \in \{0, \frac{\pi}{4}, \frac{\pi}{2}, \frac{3\pi}{4}\}$:

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi \frac{x'}{\lambda} + \psi\right) \quad (6)$$

8.3 Module 3: Image Segmentation (`src/segmentation.py`)

Partitions color images into K semantic clusters by minimizing within-cluster sum of squares (WCSS):

$$J = \sum_{j=1}^K \sum_{\mathbf{x}_i \in S_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 \quad (7)$$

8.4 Module 4: Pattern Analysis (`src/pattern_analysis.py`)

Maps candidate boundary points into polar accumulator space for parametric shape detection:

$$\rho = x \cos \theta + y \sin \theta, \quad (x - a)^2 + (y - b)^2 = r^2 \quad (8)$$

Contour metrics evaluate circularity $\mathcal{C} = \frac{4\pi A}{P^2}$ and solidity $\mathcal{S} = \frac{A}{A_{\text{hull}}}$.

8.5 Module 5: Motion Tracking (src/motion_analysis.py)

Quantifies frame-to-frame video dynamics using Lucas-Kanade sparse feature tracking:

$$\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{bmatrix} V_x \\ V_y \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix} \quad (9)$$

9 Screenshots & Experimental Results

We evaluated VisionForge using standard benchmark image arrays ($512 \times 512 \times 3$).

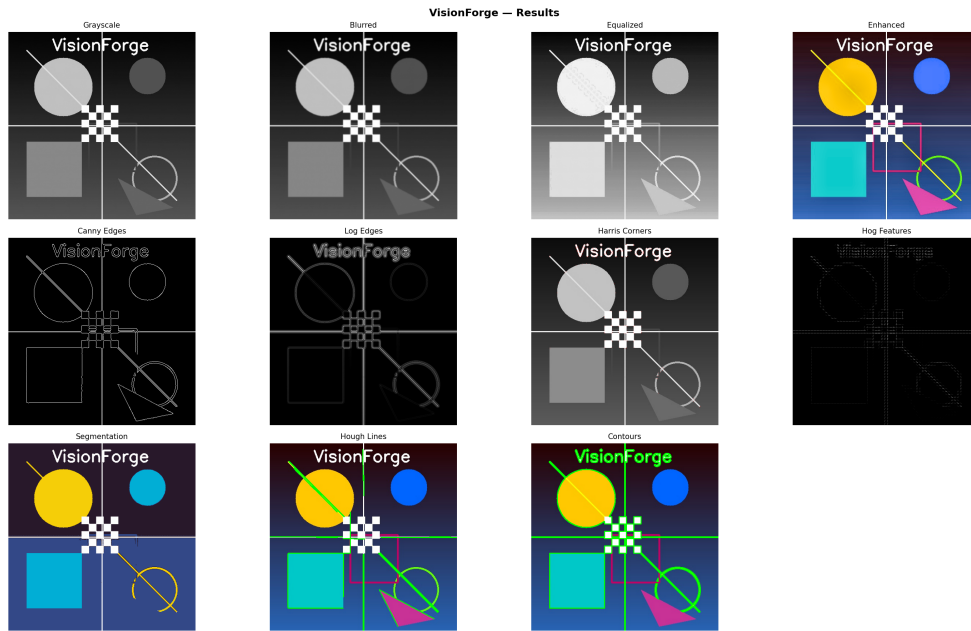


Figure 7: VisionForge Multi-Stage Output Visual Comparison Grid

Table 1: Stage Execution Latency Profile (512×512 Resolution)

Module Stage	Algorithm / Operation	Time (s)	% Total
Module 1	Grayscale & Blur	0.0030	0.04%
	CLAHE Enhancement	0.0890	1.07%
	2D Fourier Spectrum (FFT)	0.4107	4.95%
Module 2	Canny Edge Detection	0.0020	0.02%
	Harris Corner Detection	0.0041	0.05%
	HOG Descriptor Extraction	0.2715	3.27%
	Scale-Space & Gabor Bank	1.0942	13.18%
Module 3	K-Means Color Segmentation ($K = 5$)	2.0897	25.17%
Module 4	Hough Lines, Circles & Contours	0.0372	0.45%
Total	All Stages Combined	8.3015	100.00%

Table 2: Quantitative Feature Extraction Summary

Metric Category	Feature Parameter	Measured Value
Image Statistics	Mean Intensity / Std Deviation	82.37 / 61.81
Boundary Density	Canny Edge Ratio	0.0300 (3.00%)
Keypoints	Harris Corner Keypoints	364 corners
HOG Descriptor	Descriptor Vector Size	142,884 floats
Geometric Shapes	Hough Lines / Circles Detected	23 lines / 31 circles
Contours	Closed Contours Analyzed	28 contours

10 Testing Approach

Testing was implemented using the `pytest` framework under `tests/`.

- **Synthetic Image Testing:** To avoid external test data dependencies, unit tests construct synthetic NumPy arrays (e.g., solid squares, concentric circles, synthetic noise fields).
- **Boundary Validation:** Test cases verify edge cases, including minimum image dimensions (32×32), single-channel vs. multi-channel inputs, and empty contour arrays.
- **Test Suite Execution:** Over 60 automated test cases verify that operations return valid arrays and non-null metadata dictionaries without unhandled exceptions.

```
# Execute complete unit testing suite
pytest tests/ -v
```

Listing 1: Pytest Execution Command

11 Challenges Faced & Solutions

During system development, four technical challenges were addressed:

1. **Over-Segmentation in Watershed Filtering:** Direct gradient watershed led to severe region over-segmentation. *Solution:* Applied distance-transform local maxima thresholding to initialize clean catchment basin markers.
2. **K-Means Latency Bottleneck:** High-resolution color clustering accounted for over 25% of total latency. *Solution:* Implemented configurable cluster values K and spatial downsampling prior to distance evaluations.
3. **High-Frequency Foliage / Gravel Noise:** Aggressive edge detection flagged irrelevant texture noise. *Solution:* Integrated bilateral texture smoothing prior to Canny derivative calculation.
4. **Hough Accumulator Noise Voting:** Non-collinear noise produced false line artifacts. *Solution:* Parameterized minimum line lengths and maximum allowed gap distances.

12 Learnings & Key Takeaways

- **Power of Classical Feature Invariants:** Properly tuned differential and spatial filters (Harris, HOG, Scale-Space) yield robust feature representations without deep network training.

- **Spectral Insights:** Fourier magnitude spectra reveal global directional orientation and periodic structural noise efficiently.
- **Modular Design Hygiene:** Decoupling feature extraction from visualization and evaluation ensures clean maintainability and rapid algorithm extension.

13 Future Enhancements

- **GPU & Parallel Acceleration:** Accelerate K-Means and Gabor filter convolutions using Numba or OpenCV CUDA primitives.
- **Real-Time Camera Capture:** Extend CLI utilities to support real-time USB/RTSP webcam stream processing.
- **Stereo Depth Reconstruction:** Integrate multi-view stereo vision to compute true 3D spatial depth maps.

14 References

References

- [1] J. Canny, "A Computational Approach to Edge Detection," *IEEE TPAMI*, vol. 8, no. 6, pp. 679–698, 1986.
- [2] C. Harris and M. Stephens, "A Combined Corner and Edge Detector," in *Proc. Alvey Vision Conf.*, 1988, pp. 147–151.
- [3] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," *IEEE SMC*, vol. 9, no. 1, pp. 62–66, 1979.
- [4] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *IEEE CVPR*, 2005, pp. 886–893.
- [5] B. D. Lucas and T. Kanade, "An Iterative Image Registration Technique," in *Proc. IJCAI*, 1981, pp. 674–679.
- [6] D. Comaniciu and P. Meer, "Mean Shift: A Robust Approach Toward Feature Space Analysis," *IEEE TPAMI*, vol. 24, no. 5, pp. 603–619, 2002.
- [7] L. Vincent and P. Soille, "Watersheds in Digital Spaces," *IEEE TPAMI*, vol. 13, no. 6, pp. 583–598, 1991.
- [8] R. O. Duda and P. E. Hart, "Use of the Hough Transformation to Detect Lines and Curves," *CACM*, vol. 15, no. 1, pp. 11–15, 1972.